AeroCast AI

User & System Guide

Predictive airport capacity & delay-risk copilot. Forecasts hourly Arrival Acceptance Rate and severe-delay risk 3-12 h ahead for ATL & DFW. The controller decides; the model advises.

Challenge #5 | Autonomous SG 2026 · AI & Aerospace (Airbus) Track As-built guide · running copy

This guide documents the **working system** as built and verified on the provided data (148,273 flights, Jan 2023 - Jul 2025). It covers what it does, how it works, how to run it, and how to use it.

1 What it does

AeroCast AI is a human-in-the-loop decision-support tool for airport operations. For a chosen airport, at a chosen time, it answers two questions 3-12 hours ahead and presents the answers to an air-traffic controller with the evidence behind them:

- **Capacity - hourly Arrival Acceptance Rate (AAR):** how many arrivals per hour the airport can be expected to accept, with an 80% confidence band, so a controller can see a reduction coming instead of reacting when weather arrives.
- **Risk - severe-delay flagging:** which upcoming arriving flights are most likely to be delayed more than 45 minutes, as a ranked, probability-scored review list.

Every prediction carries its **weather drivers**, a **confidence** level, and a one-line plain-English **brief**. A deterministic pipeline (ML + statistics + rules) produces the numbers; a language model only explains them. The controller makes the final call.

What it actually achieves on this data (verified, honest)

Prediction	Result on the 2025 hold-out	Read
AAR (regression)	RMSE 0.47 vs climatology 0.47 (skill ~ 0)	Matches the climatology baseline - see note below
Delay risk (ranking)	ROC-AUC 0.60, PR-AUC 0.19 (base rate 0.12)	~1.6x better than the base rate at separating flights
Delay risk (operating point)	Review top 20% -> catch 33% of severe delays	1.7x lift over reviewing at random
Calibration	Brier 0.106 (Platt-calibrated)	Probabilities read as real likelihoods
Generalisation (leave-one-airport-out)	Train DFW -> test ATL ROC 0.55; train ATL -> test DFW 0.59	Beats chance across different runway geometries

The honest ceiling (state this in a demo - it is a strength)

The absolute predictive signal is **modest**, and that is a property of this dataset, not the pipeline: the file is a **~0.5% sample**, ATL and DFW see **little severe weather**, and arrival delay is **propagation-dominated** (late inbound aircraft) - not a weather feature. So AAR counts are schedule-driven (the climatology tie) and weather skill is capped. The same pipeline on the full feed at storm-prone hubs would show stronger signal. Knowing and showing this is exactly what the rubric rewards under *honest evaluation* and *trust*.

2 How it works

AeroCast runs as a **leakage-safe forecasting loop**. The rule that keeps it honest: a prediction made at time *t* for a target hour *h* (3-12 h later) uses **only data knowable at *t*** - weather observations already on record, the schedule, and climatology - never the weather measured at hour *h*. A unit test enforces this.

The right tool for each job. Machine learning and statistics do the predicting; rules and trigonometry do the physics; a language model only explains verified numbers. It never computes a value or takes an action.

The pipeline, module by module

Module (file)	What it does
Ingest `ingest.py`	Loads the CSV, parses HHMM clock fields into real datetimes, builds a continuous hourly airport-weather table and a flight table.
Enrich `enrich.py`	Fetches REAL hourly wind direction / speed / gusts from the Open-Meteo public archive (free, no key), caches it, and joins it on. This is the 'automatic public-dataset processing' step. Falls back to synthetic offline.
Features `features.py`	Computes crosswind / headwind / tailwind vs each runway heading (ATL 080, DFW 180), and lags every weather field 3/6/12 h - the leakage guard. Adds schedule, origin-weather (masked for short flights) and carrier features.
Targets `targets.py`	Builds the AAR proxy (arrivals per airport-hour) and the per-flight severe-delay label (arrival delay > 45 min).
Models `model_aar.py / model_risk.py`	Gradient-boosted regressor (AAR + interval) and classifier (delay risk), preferring XGBoost, falling back to scikit-learn. The classifier is Platt-calibrated on a validation slice.
Evaluate `evaluate.py`	Persistence + climatology baselines, RMSE / PR-AUC / lift / Brier, a leave-one-airport-out generalisation check, a failure case, and permutation feature-importances.
Copilot `copilot.py`	Turns verified output into top drivers, a confidence tier and a plain-English brief. Uses Claude if ANTHROPIC_API_KEY is set, else a template.
Dashboard `app.py`	The Streamlit control-room UI that reads the artifacts and presents everything to the controller.
Orchestrator `pipeline.py`	One entry point: ingest -> enrich -> features -> targets -> train -> evaluate -> write artifacts.

Trust built in: every forecast ships with baselines to beat, a confidence band, calibrated probabilities, the weather drivers behind it, a generalisation check, and a shown failure case - and the human stays in control.

3 How to run it

Requires Python 3.11+. The data file must be at `sources/Flight_Data_With_Weather_Final.csv`.

Install, run the pipeline, launch the dashboard

```
pip install -r requirements.txt

python -m aerocast.pipeline # trains + writes ./artifacts (~15 s)

streamlit run app.py # opens the dashboard on :8501
```

Or do all three with one command:

```
./run.sh
```

- **First run fetches wind** from Open-Meteo and caches it to `artifacts/wind_*.csv`, so later runs are instant. Offline, it falls back to a labelled synthetic wind direction and still runs end-to-end.

- **Minimal install:** runs with just `pandas numpy scikit-learn joblib requests`; `xgboost / streamlit / plotly / meteostat / anthropic` are optional upgrades.
- **Live LLM brief:** set `ANTHROPIC_API_KEY` to turn the copilot's plain-English brief from template to live Claude; the code path is ready.

Tests

```
python tests/test_pipeline.py # 5 tests: leakage guard, crosswind, parsing, labels
```

Config knobs

All in `aerocast/config.py`: **AIRPORTS** + **RUNWAY_HEADING** (add an airport in one line), **LEADS** (forecast horizons), **BUCKET** (AAR aggregation; `'1h'` default), **TEST_START** (temporal split), **SEVERE_DELAY_MIN** (delay threshold).

Artifacts land in `./artifacts`: metrics.json, aar_predictions.csv, risk_predictions.csv, drivers.json, context.json, the cached wind, and the saved models.

4 How to use it (the dashboard)

Open the dashboard (`streamlit run app.py`, then browse to `localhost:8501`). Top to bottom:

Panel	What it shows / how to read it
Honesty note	States the data caveats and the real result up front, so no one is misled.
Four metric cards	AAR RMSE vs climatology; risk PR-AUC; risk recall at the top-20% review budget.
Controls	Pick an **airport** (ATL / DFW) and drag the **day** slider across the 2025 hold-out.
AAR forecast chart	Orange = forecast, dotted grey = typical (climatology), amber band = 80% interval, blue dots = actual arrivals. A forecast dipping below typical is a capacity-reduction signal.
Capacity-reduction alert	Fires (red) when the forecast drops materially below typical in the window, with the earliest hour - the controller's early warning.
Flagged flights table	Arriving flights ranked by severe-delay probability (carrier, flight, origin, risk). This is the ‘who to review’ list; by design you review the top ~20%.
Top drivers	The weather / schedule features behind the capacity and the delay-risk predictions - the ‘why’.
Controller brief	One plain-English sentence summarising the hour: forecast capacity, drivers, how many flights flagged, confidence, and the suggested action.
Honest-evaluation panel	Expandable: the baselines, calibrated risk metrics, per-airport numbers, the leave-one-airport-out generalisation, and the worst-error failure case.

How a controller would act

- Pick your airport and look 3-12 h out. If a **capacity-reduction alert** fires, you have lead time to meter arrivals or adjust the acceptance rate before aircraft are airborne and holding.
- Scan the **flagged-flights** list for the upcoming hours; these are the arrivals worth pre-emptive attention (re-sequencing, gate/stand planning, passenger-connection warnings).
- Read the **drivers** and **brief** to understand *why*, check the **confidence**, and - because the model advises rather than decides - make the call yourself.

Built and verified on the provided data; every number here comes from the pipeline's own hold-out evaluation. See README.md for full technical detail.